



MarDRe: User's Guide

Authors:
Roberto R. Expósito, Jorge Veiga, Jorge González-Domínguez
and Juan Touriño

October 25, 2026



Contents

1 Introduction **3**
1.1 Citation . . . 3

2 Prerequisites **3**

3 Execution **4**
3.1 Command-line arguments . . . 4
3.2 Examples . . . 5
3.3 Selection of the reads . . . 5
3.4 Read names . . . 6
3.5 Optical duplicates . . . 6
3.6 Compression support . . . 6
3.7 Configuration . . . 7

4 Compilation **8**

5 Contact **9**

1 Introduction

MarDRe [1, 2] is a *de novo* MapReduce-based parallel tool to remove duplicate and near-duplicate DNA reads through the clustering of single-end or paired-end sequences from FASTQ/FASTA datasets. Duplicate reads can be seen as identical or nearly identical sequences with some mismatches. Depending on the application scenario, duplicate or near-duplicate reads do not provide any interesting biological information but can increase memory requirements and computational time of downstream analysis. This tool allows researchers and bioinformatics to avoid the analysis of not necessary reads, reducing the time of subsequent procedures with the dataset (e.g., assemblies, mappings, etc.).

MarDRe is the Big Data counterpart of ParDRe [3, 4], which employs HPC technologies (i.e., hybrid MPI/multithreading) to reduce runtime on multicore systems. Instead, MarDRe takes advantage of the MapReduce programming model originally developed by Google [5] to significantly improve ParDRe performance on distributed systems, specially on cloud-based infrastructures. Written in pure Java to maximize cross-platform compatibility, MarDRe is built upon the open-source Apache Hadoop project [6], the most popular distributed computing framework for scalable Big Data processing.

This tool is distributed as free software and is publicly available at [2] under the GPLv3 license [7]. The source code and the releases are also available at GitHub [8].

1.1 Citation

If you have used MarDRe in your research, please cite our work using reference [1].

2 Prerequisites

In order to use MarDRe, the prerequisites are:

1. Make sure you have Java Runtime Environment (JRE) version 11 or above
2. Make sure you have a working Hadoop distribution version 3.3.6 (the one MarDRe is built and tested with) [9]
   - HADOOP_HOME environmental variable must be set accordingly
3. Untar the downloaded MarDRe distribution
   - On Linux/Mac OS, just follow the instructions below:

```
[user@host ~]$ tar xzf MarDRe-v2.0.tar.gz
```

   - Alternatively use your preferred archive extraction tool
4. Set MARDRE_HOME and PATH environmental variables
   - On Linux/Mac OS you can set them in your profile or your shell configuration files (e.g., .bashrc). Follow the instructions below:

```
[user@host ~]$ export MARDRE_HOME=/path/to/mardre
[user@host ~]$ export PATH=$MARDRE_HOME/bin:$PATH
```

3 Execution

MarDRe can be executed by using the provided *mardrerun* command, which launches the MapReduce jobs to the Hadoop cluster. The only compulsory argument is the input dataset for single-end executions, or two datasets in case of paired-end executions.

The input FASTQ/FASTA datasets must be stored in the Hadoop Distributed File System (HDFS) [10]. MarDRe supports compressed datasets (see Section 3.6 for more information about this topic) and uses the Hadoop Sequence Parser (HSP) library [11], version 2.0.0, to read the input datasets. The reads are not written to the output in the order of the input datasets, and their names are kept as they are in the input (see Section 3.4).

3.1 Command-line arguments

The available command-line arguments are:

- -i. Compulsory both in single-end and paired-end scenarios. String with the HDFS path to the input sequence file in FASTA/FASTQ format.
- -p. Compulsory only in paired-end scenarios. String with the HDFS path to the second input sequence file in FASTA/FASTQ format.
- -o. Optional. String with the file name for the first output file in FASTA/FASTQ format. The default value is the same as the first input file name followed by `.NonDup`.
- -r. Optional. String with the file name for the second output file in FASTA/FASTQ format in paired-end scenarios. The default value is the same as the second input file name followed by `.NonDup`.
- -q. Specify that input sequence files are in FASTQ format. By default, MarDRe tries to autodetect the input format, but if input files are compressed the user must specify the appropriate argument.
- -f. Specify that input sequence files are in FASTA format. By default, MarDRe tries to autodetect the input format, but if input files are compressed the user must specify the appropriate argument.
- -m. Optional. Integer with the number of allowed mismatches to identify two reads as equivalent. The default value is 0.
- -l. Optional. Integer with the prefix length. During the mapping phase, MarDRe emits as key the first l encoded bases of each read. Then, reads of the same key are compared in the reducing phase. Higher prefix-length usually leads to shorter computation but can miss some duplicates. The loss of accuracy is observed when removing near-duplicate reads. Reads in the same group have exactly the same prefix (i.e., mismatches are not allowed in the prefix). The longer the prefix, more near-duplicate can be missed. Lets use as example an scenario where we try to compress near-duplicate reads with up to one mismatch. If we use a prefix of length 20 to compare two reads that only have one mismatch in position 16, MarDRe stores them in different groups, they are never compared, and both reads will be in the

output. Otherwise, with a prefix of length 15 MarDRe compares them and, as only one base is different, discards one of them. The default value is 20.

- -c. Optional. Integer with the number of bases to compare for each read (starting from the beginning of the sequence). It must be equal or greater than the prefix length and equal or less than the sequence length. The default value is equal to the sequence length (i.e., all bases are compared).
- -d. Optional. Integer with the maximum Euclidean distance between two duplicate reads to consider them optical duplicates. If it is specified, MarDRe only removes the optical duplicates, that is, duplicate reads of the same tile whose distance is at most this value, according to the *:X:Y* coordinates of their names. By default, all the duplicates are removed. See Section 3.5 for more information about this topic.
- -nr. Optional. Integer with the number of reducers. The default value is 1.
- -v. Print out the version of the program and exit.
- -h. Print out the usage of the program and exit.

The generic options of Hadoop (e.g., `-D property=value`) can also be used, but they must be specified before the arguments of MarDRe.

3.2 Examples

The following command removes the duplicates of identical reads (no mismatches) of a single-end dataset using a prefix length of 15 and 8 reducers:

```
[user@host ~]$ mardrerun -i dataset.fastq -l 15 -nr 8
```

The following command shows a similar example but for paired-end execution, allowing two mismatches and using the default prefix length (20) and 8 reducers:

```
[user@host ~]$ mardrerun -i dataset1.fastq -p dataset2.fastq -m 2
    -nr 8
```

The following command only removes the optical duplicates that are at most 100 units apart of a single-end dataset:

```
[user@host ~]$ mardrerun -i dataset.fastq -d 100 -nr 8
```

3.3 Selection of the reads

MarDRe groups the reads that have the same prefix and, inside each group, keeps only one read of each set of duplicate or near-duplicate reads, that is, reads with at most m mismatches in the compared bases. The read that is kept is the one with the highest average quality score. If several reads have the same average quality, the one with the lowest name is kept, so the result does not depend on the order in which the reads arrive. In paired-end scenarios the quality of a pair is the sum of the average qualities of its two

reads and the name of the first read decides the ties. FASTA reads have no quality scores, so the one with the lowest name is kept.

If the number of allowed mismatches is 0, the reads that are kept are always the same. Otherwise (and also when using the option -d), each read is compared with the reads that are already kept, in the order in which they arrive, and it replaces the one that was kept if it has a higher quality. Therefore, the result can depend on the order in which the reads arrive to the reducers, and a small number of near-duplicates (about 0.01% of the reads in our tests) can remain in the output.

3.4 Read names

The name of each read is written to the output exactly as it is in the input dataset. MarDRe 1.x cut the names at their first space. This behavior can be restored with the generic Hadoop option `-Dhsp.trim.sequence.name=true`, which makes the output smaller and the execution slightly faster:

```
[user@host ~]$ mardrerun -Dhsp.trim.sequence.name=true -i dataset
   .fastq -nr 8
```

This option must not be used together with -d, because the coordinates of the reads can be in the part of the name that is removed.

3.5 Optical duplicates

Optical duplicates are reads that are detected more than once by the sequencer because they are very close in the flow cell. The option -d makes MarDRe only remove the duplicate reads (according to the criteria of the previous sections) that, in addition, are in the same tile and at most the given Euclidean distance apart. The distance is computed in the units of the coordinates of the names and it is rounded down.

The coordinates are taken from the name of each read: the first word of the name that ends in two numeric fields (*:X:Y*, with up to nine digits each), which can be followed by text that is not a number, such as */1*. What precedes the coordinates in that word identifies the tile, so two reads must have the same one to be optical duplicates. The following names are two examples, one from Illumina and the other from the SRA:

```
@K00417:10:HHKL2BBXX:1:1101:1000:2000 1:N:0:AAAA
@DRR055516.1 K00211:24:H3MWGBBXX:7:1101:1406:1334/2
```

If the name of a read does not have coordinates, the execution fails with an error that names the read. In paired-end scenarios the coordinates of the first read of each pair are used. The names must not be cut with `-Dhsp.trim.sequence.name=true`.

3.6 Compression support

MarDRe supports the processing of input datasets compressed with Gzip (i.e., .gz extension) and BZip2 (i.e., .bz2 extension) codecs. However, when considering compressed data that will be processed by Hadoop, it is important to understand whether the underlying compression format supports splitting, as many codecs need the whole input stream to uncompress successfully.

On the one hand, it is impossible to start reading at an arbitrary point in a Gzip file and therefore impossible for a map task to read its input split independently of the others. For this reason, Gzip does not support splitting and Hadoop will not split the gzipped input dataset. This will work, but at the expense of performance: a single map will parse the whole input dataset, which prevents parallelism during the parsing of the reads during the map phase, while the duplicate removal step can still be performed in parallel during the reduce phase. In terms of performance, it would be probably better to first uncompress the input dataset before storing it in HDFS. On the other hand, BZip2 does compression on blocks of data and later these compressed blocks can be decompressed independent of each other, so it does support splitting. Therefore, BZip2 is the recommended codec to use with Hadoop for best performance and parallelism, although in paired-end scenarios each BZip2 file of a pair is read in a single split, which reduces the parallelism of the map phase. Note that if you are using compressed input datasets, MarDRe will also compress the output datasets using the same codec.

Finally, even if you are using uncompressed input datasets, Hadoop may benefit from compressing the intermediate output of the map phase in both single-end and paired-end modes. Since the map output is written to disk and transferred across the network to the reducer nodes, by using a fast compressor such as snappy, you may get performance gains simply because the volume of network data to transfer is reduced. Moreover, the output files of the intermediate MapReduce jobs needed in paired-end mode when using a reduce-side join (see next section) can also be compressed, thus reducing disk I/O overhead between jobs. MarDRe supports both types of intermediate compression using the snappy codec, which can be configured by setting COMPRESS_MAP_OUTPUT and COMPRESS_INTERMEDIATE_OUTPUT options, as shown next.

3.7 Configuration

MarDRe can be configured by means of the *mardre.conf* file located at the *etc* directory. The available parameters are:

- MERGE_OUTPUT (boolean). Merge output files into a single file stored in HDFS. The default value is false.
- DELETE_TEMP (boolean). Delete intermediate files created by Hadoop (if any). The default value is true.
- PAIRED_END_MAP_JOIN (boolean). Enable the use of a map-side join in paired-end scenarios when set to true, otherwise a reduce-side join is performed. The default value is true, which usually leads to a better performance.
- COMPRESS_MAP_OUTPUT (boolean). Enable the compression of the intermediate map output using the snappy codec. It requires the snappy library installed on the system and the Hadoop native library (i.e., libhadoop) compiled with snappy support. The default value is false.
- COMPRESS_INTERMEDIATE_OUTPUT (boolean). Enable the snappy compression of the intermediate output files in paired-end scenarios using a reduce-side join (the same requirements apply as before). The default value is false.

- INPUT_BUFFER_SIZE (integer). The buffer size in bytes that is used for input read operations. It should probably be a multiple of the hardware page size (e.g., 4096). The default value is 65536 bytes.
- IN_MAPPER_COMBINER (boolean). Enable the use of the in-mapper combiner when possible. The default value is false.
- IN_MAPPER_COMBINER_CACHE_SIZE (integer). Maximum number of entries cached by the in-mapper combiner. The default value is 65536 entries.
- CLUSTER_LIST_INITIAL_CAPACITY (integer). Initial capacity of the ArrayList used to store reads during duplicate removal. The default value is 65536 entries.
- HDFS_BASE_PATH (string). Base path on HDFS where MarDRe stores the output files as well as temporary intermediate files. The user running MarDRe must have write permissions on this path. The default value is blank, which means to use the HDFS user's home directory.
- HDFS_BLOCK_REPLICATION (short). HDFS block replication factor for output sequence files. The default value is 1.

The *mardre.conf* file is only read by the command that launches the job, so MARDRE_HOME does not need to be set in the cluster nodes. The diagnostics of MarDRe are reported through the SLF4J API: the final summary and timings are printed on the standard output, while the rest of messages follow the logging configuration of Hadoop (e.g., on the standard error when launching the job). The statistics of each task are reported in the log of its container.

4 Compilation

In case you need to recompile the MarDRe distribution, the prerequisites are:

1. Make sure you have Java Development Kit (JDK) version 11 or above (MarDRe is tested with JDK 11, 17 and 21)
2. Make sure you have a working Apache Maven distribution version 3 or above
   - https://maven.apache.org/install.html
3. Make sure you have the Hadoop Sequence Parser (HSP) library [11] version 2.0.0 installed in your Maven local repository. It is not published in any public Maven repository, so it is a requirement to install it before building MarDRe (the distributed JAR already contains it):
   - In order to download, compile, build and install HSP, just execute the following commands:

```
[user@host ~]$ git clone --branch v2.0.0 https://github.
   com/UDC-GAC/hsp.git
[user@host ~]$ cd hsp
[user@host hsp]$ mvn install
```

In order to build the JAR distribution, just execute the following Maven command from within the MarDRe root directory:

```
[user@host mardre]$ mvn package
```

The first time you execute this command, Maven will download all the plugins and related dependencies it needs to fulfill the command. From a clean installation of Maven, this can take quite a while. If you execute the command again, Maven will now have what it needs, so it will be able to execute the command much more quickly.

This command also runs the tests of MarDRe, which execute the whole tool on the local runner of Hadoop and take about a minute. They can be skipped by adding `-DskipTests` to the command. The resulting file *target/MarDRe-bin.jar* contains MarDRe and all its dependencies except Hadoop.

5 Contact

MarDRe has been developed in the Computer Architecture Group [12] at the University of A Coruña [13] by the following authors:

- Roberto R. Expósito: http://gac.udc.es/~rreye
- Jorge Veiga: http://gac.udc.es/~jveiga
- Jorge González-Domínguez: http://gac.udc.es/~jgonzalezd
- Juan Touriño: http://gac.udc.es/~juan

To report any question, bug, requirement or information about MarDRe, feel free to contact us at [2].

References

[1] Roberto R. Expósito, Jorge Veiga, Jorge González-Domínguez, and Juan Touriño. MarDRe: efficient MapReduce-based removal of duplicate DNA reads in the cloud. Bioinformatics. *Bioinformatics*, 33(17):2762–2764, 2017.

[2] MarDRe webpage. https://mardre.des.udc.es.

[3] Jorge González-Domínguez and Bertil Schmidt. ParDRe: faster parallel duplicated reads removal tool for sequencing studies. *Bioinformatics*, 32(10):1562–1564, 2016.

[4] ParDRe webpage. https://sourceforge.net/projects/pardre.

[5] Jeffrey Dean and Sanjay Ghemawat. MapReduce: simplified data processing on large clusters. *Communications of the ACM*, 51(1):107–113, 2008.

[6] Apache Hadoop. http://hadoop.apache.org.

[7] GNU General Public License version 3 (GPLv3). https://www.gnu.org/licenses/gpl-3.0.en.html.

[8] MarDRe GitHub repository. https://github.com/UDC-GAC/MarDRe.

[9] Apache Hadoop Cluster Setup. http://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-common/ClusterSetup.html.

[10] Konstantin Shvachko, Hairong Kuang, Sanjay Radia, and Robert Chansler. The Hadoop distributed file system. In *Proceedings of the 26th IEEE Symposium on Mass Storage Systems and Technologies (MSST'10)*, pages 1–10, Incline Village, NV, USA, 2010.

[11] HSP: Hadoop Sequence Parser library for FASTQ/FASTA datasets. https://github.com/UDC-GAC/hsp.

[12] Computer Architecture Group. http://gac.udc.es/english.

[13] University of A Coruña. http://www.udc.gal/index.html?language=en.